Coding: Clean Architecture

CleanArchitecture

```
---
applyTo: '**/*.{csproj,cs,json}'
---

# Clean Architecture Standards for C# Solutions

This document defines industry standards and best practices for implementing Clean Architecture in C# solutions, with a focus on maintainability, testability, and separation of concerns.

![Clean Architecture](https://blog.cleancoder.com/uncle-bob/images/2012-08-13-the-clean-architecture/CleanArchitecture.jpg)

## Key Principles

- **Separation of Concerns**: Organize code into distinct layers (Domain, Application, Infrastructure, Presentation) to isolate business logic from external concerns.
- **Dependency Rule**: Dependencies always point inward. Outer layers (UI, Infrastructure) depend on inner layers (Domain, Application), never the reverse.
- **Domain-Centric Design**: The Domain layer contains business logic and entities. It should have no dependencies on other layers.
- **Use Cases**: The Application layer orchestrates business rules and coordinates domain entities to fulfill use cases.
- **Interface Abstraction**: Define interfaces in inner layers; implement them in outer layers. Use Dependency Injection to invert dependencies.
- **Infrastructure Isolation**: Infrastructure code (e.g., data access, external services) is isolated and replaceable.
- **Testability**: Business logic is testable in isolation, without reliance on infrastructure or UI.

## Recommended Folder Structure

```
/src
  /Domain (Optional - only if there is business logic, no anemic domains)
  /Application
  /Infrastructure

  ** at least one (API or Observer or Daemon)
  /API (Optional)
  /Observer (Optional)
  /Daemon (Optional)

  /Events (Optional - if application is producing events)
```

- Place core business logic in `Domain`.
- Use `Application` for use cases, services, and interfaces.
- Implement external concerns in `Infrastructure`.
- `API/Observer/Daemon` contains either an API, observer (e.g., event listeners), or background services.
- A domain that has an API, Observer and Daemon, ensure that they share the same infrastructure, domain and application layer.
- Daemon - another term for background services or long-running processes.

Coding Standards

- Use clear, descriptive names for layers, projects, and files.
- Avoid circular dependencies between projects or namespaces.
- Favor composition over inheritance.
- Apply SOLID principles throughout all layers.
- Keep controllers and UI thin; delegate logic to the Application layer.
- Use Dependency Injection for all external dependencies.
- Write unit tests for business logic in the Domain and Application layers.
- Do not provide explicit technology solution names in interface definitions.
- If the return type is not clear, do not use `var` but the explicit type returned.
  ```csharp
  var date = new DateTime(); // this is fine - we can infer that the date type is DateTime
  var foo = GetItem(); // this is not fine - we cannot infer the type of foo
  Foo foo = GetItem(); // this is fine - we explicitly declare the type of foo
  ```
- Avoid anemic domain models by ensuring entities have behavior and not just data. (if there is no behavior, do not include domain layer)
- Do not leak infrastructure layer details into the domain, application layers or in method signatures.
- Do not leak application layer details into the domain layer or in method signatures.
- Avoid redundant code and duplication across layers.
- Handle exceptions with an exception middleware to avoid duplication of logging and error handling logic.
- Exception Middleware does not return exception details to the client.

Project Dependency Guidelines

```

- **Direction of Dependencies**: Projects must reference only those in inner layers. For example, `Presentation` can reference `Application`, which can reference `Domain`, but never the reverse.
- **No Reverse References**: Inner layers (Domain, Application) must not reference outer layers (Infrastructure, Presentation).
- **Interface Placement**: Define interfaces in inner layers (Domain, Application) and implement them in outer layers (Infrastructure, Presentation).
- **Infrastructure Isolation**: Infrastructure should depend on abstractions from Application or Domain, not the concrete implementations of those layers.
- **Avoid Circular Dependencies**: Never create project references that result in circular dependencies between layers.
- **Dependency Injection**: Use DI to provide implementations from outer layers to inner layers at runtime, without direct references.

## Additional Recommendations

### Example Project Structure

A typical Clean Architecture solution may be organized as follows:

```
/AI.KnowledgeBaseAndStandards
  /src
    /Domain (Optional - only if there is business logic, no anemic domains)
      - Entities/
      - ValueObjects/
      - DomainEvents/
      - Interfaces/

    /Application
      - UseCases/
      - Services/ (Optional - for utility classes)
      - Interfaces/
        - Infra/ (For infrastructure-related interfaces)
      - DTOs/

    /Infrastructure
      - Models/
      - Mappers/ (Optional)
      - ExternalServices/
      - Repositories/

    ** at least one (API or Observer or Daemon)
    - API (Optional)
      - Controllers/
      - DTOs/
        - Requests/
        - Responses/
      - Filters (Optional)
      - Mappers/ (Optional)
      - Middlewares (Optional)
    - Observer (Optional)
      - Handlers/
    - Daemon (Optional)

    /Events (Optional - if application is producing events)
```

Interface Naming Conventions

- Prefix interfaces with `I` (e.g., `IRepository`, `IService`).
- Use descriptive names that reflect the contract's purpose.
- Application layer will have use cases, starts with a verb in present tense followed by a noun and a `Handler` suffix
  - `I{ActionItem}Handler` (e.g., `IPlaceOrderHandler`)

Observer Naming Conventions

- User `Handlers` folder root project
- Prefix handlers with event name (e.g., `OrderPlacedEventHandler`)
- Suffix must end in EventHandler

Event Ownership

- Events should be owned by the domain that produces it
- Ensure event names have a suffix reflecting a verb in the past tense and they end in `Event` (e.g., `OrderPlacedEvent`)
- Events should live in their own project that generates a nuget package and has semantic versioning(e.g. UPX.Provider.Events)

```

### Dependency Injection Guidance

- Use Microsoft.Extensions.DependencyInjection or similar DI frameworks.
- Register implementations in the outermost layer (e.g., Infrastructure, Presentation).
- Example registration:
  ```csharp
  services.AddScoped<IRepository, EfRepository>();
```
```

DTO Mapping Guidelines

- Create an explicit mapper class or use explicit operator.
- For simple mapping use the explicit operator.
- For the complex mapping, create an explicit mapper class with extension methods.
- Example for the using explicit operator for simple mapping:

  ```csharp
    using UPX.Patient.Application.DTOs;

    namespace UPX.Patient.API.DTOs.Responses
    {
        public class APIResponse
        {
            public string firstName { get; set; }
            public string lastName { get; set; }
            public static explicit operator APIResponse(ApplicationResponseDto payload)
            {
                return new APIResponse
                {
                    firstName = payload.FirstName,
                    lastName = payload.LastName
                };
            }
        }
    }
  ```

- Example for using explicit mapper class for complex object mapping using extension methods:

  ```csharp
    using UPX.Patient.API.DTOs.Requests;
    using UPX.Patient.Application.DTOs;

    namespace UPX.Patient.API.Mappers
    {
        public static class APIMappers
        {
            public static PatientDto ToPatientDto(this Patient entity)
            {
                return new PatientDto
                {
                    Id = entity.Id,
                    FirstName = entity.FirstName,
                    LastName = entity.LastName,
                    DateOfBirth = entity.DateOfBirth,
                    Address = new AddressDto
                    {
                        Street = entity.Address.Street,
                        City = entity.Address.City,
                        State = entity.Address.State,
                        ZipCode = entity.Address.ZipCode
                    }
                };
            }
        }
    }
  ```

Infrastructure layer guidelines

- **External API integrations**: Any connections to external APIs or third-party services (typically implemented using HttpClient) should be organized in the `ExternalServices` folder.
- **Data storage connections**: Any direct connections to storage systems (e.g., Cosmos DB, Redis cache, SQL databases) should be organized in the `Repositories` folder.

Cross-Cutting Concerns

- Place cross-cutting concerns (e.g., logging, validation, error handling) in dedicated folders or projects.
- Use middleware or decorators to keep these concerns decoupled from business logic.

Common Anti-Patterns to Avoid

- **Anemic domain models** (business logic outside domain entities).

  ```csharp
  // ❌ BAD: Anemic Domain Model - Entity has only properties, no behavior
  public class Order
  {
      public int Id { get; set; }
      public DateTime OrderDate { get; set; }
      public decimal Total { get; set; }
  ```

```
      public string Status { get; set; }
      public List<OrderItem> Items { get; set; } = new();
  }
  ```

- Leaking infrastructure details into domain/application layers.
- Overuse of inheritance; prefer composition.
- Circular dependencies between layers.
- Avoid dependencies that are not fully open source or that require commercial licensing. Prefer permissive (e.g., MIT) licenses and explicit mapping over reflective mapping libraries (e.g., AutoMapper).

Unit Test Standards Reference

- See `CSharpUnitTest.instructions.md` for test structure and isolation guidelines.
- Ensure tests cover domain and application logic in isolation from infrastructure and UI.

Migration and Legacy Integration

- For legacy code or data migration, create dedicated migration services in the Infrastructure layer.
- Keep migration logic isolated from core business logic.

References

- [Clean Architecture by Robert C. Martin (Uncle Bob)](https://8thlight.com/blog/uncle-bob/2012/08/13/the-clean-architecture.html)
- [Microsoft Docs: Clean Architecture](https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures#clean-architecture)